



From Claude Beginner To Pro

CHEAT SHEET

AI Edge | The complete guide to Cowork & Claude Code

THE 3 LEVELS OF CLAUDE

LEVEL 1

Chat

You type, Claude responds

You do all the work

Input > output > repeat

Where most users live

LEVEL 2

Cowork

Your AI assistant

Handles daily tasks

Works with your files

Think: "do this for me"

LEVEL 3

Claude Code

Your AI engineer

Builds tools & systems

Creates infrastructure

Think: "build this for me"

YOUR PROJECT FOLDER

Every project gets its own folder. Claude reads these files for context before doing any work.

```
/Your-Project/  
  INSTRUCTIONS.md      <- Employee handbook  
  MEMORY.md           <- Long-term brain (the hack)  
  CONTEXT-strategy.md  <- Reference docs for the job  
  CONTEXT-brand.md     <- Additional context as needed  
  /outputs/           <- Where finished work goes
```

The more relevant context you give, the better the output. But only relevant.

INSTRUCTIONS.MD — YOUR AGENT'S EMPLOYEE HANDBOOK

The most important file. Tells Claude who it is, what it does, and how to do it well.

- **Who you are** — One line. "You are my [role] for [project]."
- **What you do** — List exact jobs. Not "help with content" — be specific.
- **Rules** — Tone, fact-checking, when to ask vs guess.
- **Good output** — Describe ideal output: length, format, style.
- **Memory rule** — "Read MEMORY.md at start. Update it at end."

Template (copy this):

```
# INSTRUCTIONS

## Who You Are
You are my [role] for [project/company].

## What You Do
- [Specific task 1]
- [Specific task 2]
- [Specific task 3]

## Rules
- Match my tone: [describe your voice]
- Never invent stats - flag anything needing a source
- If unclear, ask ONE question before starting
- When you learn something new, add it to MEMORY.md

## What Good Output Looks Like
- [Your format/length/style preferences here]

## Memory
Start of every session: read MEMORY.md for context
End of every session: update MEMORY.md with new learnings
```

*These are .md (markdown) files. Edit in any text editor. # = heading, - = list, ****text**** = bold.*

THE MEMORY HACK — THIS CHANGES EVERYTHING

The problem: Claude doesn't remember between sessions.

The fix: MEMORY.md — Claude reads at start, updates at end.

- Add to INSTRUCTIONS: "Read MEMORY.md at start. Update at end."
- Claude logs: preferences, corrections, patterns, decisions
- Every correction is written down — same mistake never happens twice
- Week 1: a few lines. Week 4: knows your entire workflow

Claude has a built-in memory feature, but it's limited. MEMORY.md gives you full control — structured, portable, works across Cowork and Codex.

MEMORY.md Template:

```
# MEMORY

## Preferences
- [Format, tone, tools, length preferences]
- [e.g. "Prefers dot-point scripts, not prose"]

## Corrections
- [Mistakes Claude made + the right answer]
- [e.g. "Don't use the word straightforward"]

## Patterns
- [Recurring tasks, common workflows]
- [e.g. "Briefs follow: hook > angle > outline"]

## Decisions
- [Choices that shouldn't be revisited]
- [e.g. "Video framed as Beginner to Pro"]
```

Don't edit this manually. Let Claude maintain it. The file compounds over time.

CONTEXT FILES — WHAT TO INCLUDE

Only include docs relevant to the agent's job. Name them clearly.

- **Content creator:** Content strategy, brand voice, past scripts, style guide
- **Business owner:** Company overview, SOPs, org chart, product docs
- **Freelancer:** Client briefs, portfolio, rate card, communication style
- **Developer:** Architecture docs, coding standards, API specs, README

Think: what would I hand a new employee on their first day?

COWORK VS CLAUDE CODE — WHEN TO USE WHICH

COWORK (Assistant)

Use when you have a TASK:

- Organise files and folders
- Clean up messy documents
- Research and summarise
- Process spreadsheets
- Draft emails, posts, copy

One-off tasks. Done once.

CLAUDE CODE (Engineer)

Use when you need a SYSTEM:

- Build tools and scripts
- Create automations
- Set up integrations
- Scaffold entire projects
- Write, test, and debug code

Permanent infrastructure. Forever.

THE PRO LOOP — HOW COWORK & CODE WORK TOGETHER

This is the real unlock. Your assistant and engineer feed each other in a compounding cycle.

1. Give Cowork a real task from your daily work
2. Notice when you keep giving the same type of task — that's a pattern
3. That's your signal — go to Claude Code
4. Code builds a system that automates that task permanently
5. Cowork's job gets easier — it now reviews instead of creates
6. Repeat. Every loop = less manual work

The Compounding Timeline:

Week 1

Handing off
individual tasks

Learning to delegate

Week 4

Systems handle
categories of work

Patterns > systems

Week 8

Agents compound
while you sleep

Infrastructure 24/7

SETTING UP COWORK — STEP BY STEP

- **1. Get it:** Download from Claude's website or your Claude Pro account
- **2. Build folder:** Create project folder with INSTRUCTIONS.md, MEMORY.md, context files
- **3. Open it:** Open Cowork. It sees your desktop and works with your files
- **4. First task:** Start with something real. Not a test. A task you'd actually do today
- **5. Check memory:** After each session, check MEMORY.md was updated. Correct anything wrong
- **6. Compound:** The more you use it, the better it gets. Week 4 is where it clicks

Cowork is best for non-technical users. If terminals scare you, start here.

SETTING UP CLAUDE CODE — STEP BY STEP

- **1. Install:** Claude Code lives in the terminal. Install via npm
- **2. Navigate:** Navigate to your project folder in terminal. Run: `claude`
- **3. Same folder:** Claude Code reads the same INSTRUCTIONS.md and context files as Cowork
- **4. First build:** Describe what you want built in plain English. Be specific on inputs/outputs

- **5. Let it work:** Code will write, test, debug, and deliver. You review the output
- **6. Feed the loop:** Every tool Code builds makes Cowork's job easier. That's the loop

You don't need to know how to code. Describe the outcome. Claude Code handles the engineering.

EXAMPLE PROMPTS TO GET STARTED

Cowork (your assistant):

```
Organise this messy notes file into ranked briefs.  
For each idea: hook, angle, difficulty (1-5),  
and rank by potential. Format as clean markdown.
```

Claude Code (your engineer):

```
Build a Python script called brief_generator.py  
that takes a title and topic, reads my  
CONTEXT-strategy.md, and outputs a formatted brief  
with hooks, angle, outline, and retention notes.  
Save as markdown in the /briefs/ folder.
```

COMMON MISTAKES TO AVOID

- **No context:** It gives generic, useless output. Set up INSTRUCTIONS.md first.
- **Skipping memory:** You lose everything between sessions. Set up MEMORY.md from day one.
- **Starting advanced:** Like managing 5 employees when you've never managed 1. Start Cowork.
- **Vague instructions:** "Help with content" tells Claude nothing. Be specific.
- **Dumping everything:** More files isn't better. Irrelevant context confuses Claude.
- **Not reviewing:** Only you can spot bad output. Review, correct, let MEMORY.md log it.
- **Set and forget:** Context files get stale. Update when strategy changes.
- **Missing patterns:** Same task 3x a week? That's a signal to build with Code.

ADVANCED TIPS

- Multiple project folders — one per client or project. Each has its own INSTRUCTIONS.md and MEMORY.md so Claude switches context cleanly.
- When MEMORY.md gets long (50+ entries), ask Claude to reorganise it — group related items, archive old decisions, keep it clean.
- INSTRUCTIONS.md can reference other files: "For style rules, see CONTEXT-brand.md." Claude reads them when relevant.
- Save the prompt that created each tool Claude Code builds. If you need to rebuild or modify later, the original prompt is your blueprint.
- Both tools come with Claude Pro. No separate subscription needed.

20-MINUTE QUICK START CHECKLIST

- ☐ Create your project folder
- ☐ Write INSTRUCTIONS.md using the template on page 1
- ☐ Create MEMORY.md with 4 sections: preferences, corrections, patterns, decisions
- ☐ Add 1-2 context files relevant to your work
- ☐ Open Cowork and give it a real task — not a test, something real
- ☐ After the session, verify Claude updated MEMORY.md correctly
- ☐ After a few sessions, notice patterns — that's when you build with Code
- ☐ Watch the loop compound. Week 1 is fine. Week 4 is where it clicks

AI Edge

Free Telegram community | [Link in the video description](#)